Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1112 / 1112	**1 / 1**
	Accuracy : 100%

Points Scored	Time Taken
2 / 2	**0.84**
Your Total Score: 45 ↑	

Solve Next

Bubble Sort | Floor in a Sorted Array | Closest Triplet

Stay Ahead With:

Java (21) | Start Timer

```java
import java.util.*;

class Solution {

    public long findMinDiff (ArrayList<Integer> a, int m) {

        Collections.sort(a);

        int n = a.size();   // get size here

        long minDiff = Long.MAX_VALUE;

        for(int i=0; i+m-1<n; i++) {

            long diff = a.get(i+m-1) - a.get(i);

            minDiff = Math.min(minDiff, diff);
        }

        return minDiff;
    }
}
```

Custom Input | Compile & Run | Submit

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1112 / 1112	**1 / 1**
	Accuracy : 100%

Points Scored	Time Taken
2 / 2	**0.3**
Your Total Score: 47 ↑	

Solve Next

Sorted subsequence of size 3 | Array Duplicates | Two Sum - Pair with Given Sum

Java (21) | Start Timer

```java
class Solution {

    public static int smallestSubWithSum(int x, int[] arr) {

        int n = arr.length;
        int start = 0;
        int sum = 0;
        int minLength = Integer.MAX_VALUE;

        for(int end = 0; end < n; end++) {

            sum += arr[end];

            // shrink window while sum > x
            while(sum > x) {

                minLength = Math.min(minLength, end - start + 1);

                sum -= arr[start];
                start++;
            }
        }

        if(minLength == Integer.MAX_VALUE)
            return 0;

        return minLength;
    }
}
```

Custom Input | Compile & Run | Submit

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1111 / 1111	**1 / 1**
	Accuracy : 100%

Points Scored	Time Taken
2 / 2	**0.22**
Your Total Score: 49 ↑	

Solve Next

Wave Array | Sort by Absolute Difference | Convert an array to reduced form

Stay Ahead With:

Java (21) | Start Timer

```java
class Solution {

    public void threeWayPartition(int[] arr, int a, int b) {

        int low = 0;
        int mid = 0;
        int high = arr.length - 1;

        while(mid <= high) {

            if(arr[mid] < a) {
                int temp = arr[low];
                arr[low] = arr[mid];
                arr[mid] = temp;
                low++;
                mid++;
            }
            else if(arr[mid] > b) {
                int temp = arr[mid];
                arr[mid] = arr[high];
                arr[high] = temp;
                high--;
            }
            else {
                mid++;
            }
        }
    }
}
```

Custom Input | Compile & Run | Submit

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1112 / 1112	**1 / 1**
	Accuracy : 100%

Points Scored	Time Taken
4 / 4	**0.5**
Your Total Score: 53 ↑	

Solve Next

Rearrange Array Alternately | Count Number | Subarray Inversions

Stay Ahead With:

Java (21) | Start Timer

```java
class Solution {
    public int minSwap(int arr[], int k) {
        int n = arr.length;

        int good = 0;
        for(int i = 0; i < n; i++){
            if(arr[i] <= k) good++;
        }

        int bad = 0;
        for(int i = 0; i < good; i++){
            if(arr[i] > k) bad++;
        }

        int ans = bad;

        int i = 0, j = good;
        while(j < n){
            if(arr[i] > k) bad--;
            if(arr[j] > k) bad++;
            ans = Math.min(ans, bad);
            i++;
            j++;
        }

        return ans;
    }
}
```

Custom Input | Compile & Run | Submit

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1115 / 1115	**1 / 1**
	Accuracy : 100%

Points Scored	Time Taken
1 / 1	**0.59**
Your Total Score: 54 ↑	

Solve Next

Multiply Array | Mean of an Array | Greatest of three numbers

Stay Ahead With:

Java (21) | Start Timer

```java
import java.util.*;

class Solution {
    public double findMedian(int[] arr) {
        Arrays.sort(arr);
        int n = arr.length;

        if(n % 2 == 1) {
            return arr[n/2];
        } else {
            return (arr[n/2] + arr[n/2 - 1]) / 2.0;
        }
    }
}
```

Custom Input | Compile & Run | Submit

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1115 / 1115	**1 / 2**
	Accuracy : 50%

Points Scored	Time Taken
4 / 4	**2.05**
Your Total Score: 58 ↑	

Solve Next

Find kth element of spiral matrix | Rotate by 90 degree | Reverse Spiral Form of Matrix

Java (21) | Start Timer

```java
    public ArrayList<Integer> spirallyTraverse(int matrix[][]) {
        ArrayList<Integer> res = new ArrayList<>();

        int r = matrix.length;
        int c = matrix[0].length;

        int top = 0, bottom = r - 1;
        int left = 0, right = c - 1;

        while(top <= bottom && left <= right) {

            for(int i = left; i <= right; i++)
                res.add(matrix[top][i]);
            top++;

            for(int i = top; i <= bottom; i++)
                res.add(matrix[i][right]);
            right--;

            if(top <= bottom) {
                for(int i = right; i >= left; i--)
                    res.add(matrix[bottom][i]);
                bottom--;
            }

            if(left <= right) {
                for(int i = bottom; i >= top; i--)
                    res.add(matrix[i][left]);
                left++;
            }
        }

        return res;
    }
}
```

Custom Input | Compile & Run | Submit

leetcode.com/problems/search-a-2d-matrix/submissions/1926424997/

Problem List | Submit | 0 | Premium

Description | Accepted | Editorial | Solutions | Submissi

Submit Ctrl Enter

All Submissions

Accepted 133 / 133 testcases passed

NITIN Kumar submitted at Feb 21, 2026 20:20

Editorial | Solution

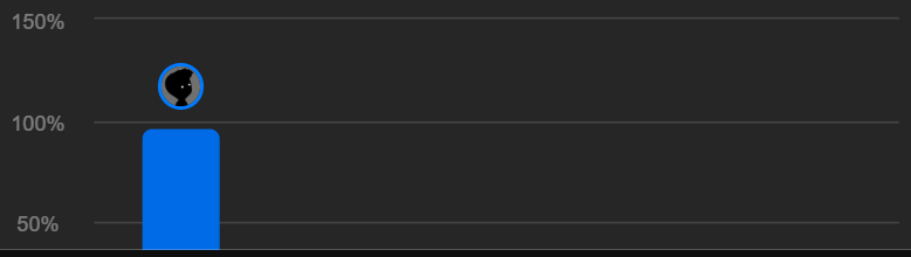
Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

44.00 MB | Beats 42.07%



Java | Auto

```
            else if(matrix[row][col] < target)
                low = mid + 1;
            else
                high = mid - 1;
        }

        return false;
    }
}
```

Saved | Ln 27, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

matrix =

Median in a row-wise sorted Matrix

Difficulty: **Medium** Accuracy: **55.05%** Submissions: **172K+** Points: **4**

Given a **row-wise sorted** matrix **mat[][]** of size n*m, where the number of rows and columns is always **odd**. Return the **median** of the matrix.

Examples:

Input: mat[][] = [[1, 3, 5],
[2, 6, 9],
[3, 6, 9]]

Output: 5

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1117 / 1117

Attempts : Correct / Total

1 / 1

Accuracy : 100%

Java (21) | Start Timer

```java
            min = Math.min(min, mat[i][0]);
            max = Math.max(max, mat[i][c-1]);
        }

        int desired = (r * c + 1) / 2;

        while(min < max){
            int mid = min + (max - min) / 2;
            int place = 0;

            for(int i = 0; i < r; i++){
                place += upperBound(mat[i], mid);
            }

            if(place < desired)
                min = mid + 1;
            else
                max = mid;
        }

        return min;
    }

    int upperBound(int[] arr, int target){
        int low = 0, high = arr.length;
        while(low < high){
            int mid = (low + high) / 2;
            if(arr[mid] <= target)
                low = mid + 1;
            else
                high = mid;
        }
        return low;
    }
}
```

Custom Input | Compile & Run | Submit

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1111 / 1111	**1 / 1**
	Accuracy : 100%

Points Scored	Time Taken
4 / 4	**0.74**
Your Total Score: 66 ↑	

Solve Next

Max sum in the configuration | Boolean Matrix | Row with Minimum 1s

Stay Ahead With:

Java (21) | Start Timer

```java
class Solution {
    int rowWithMax1s(int arr[][]) {

        int n = arr.length;
        int m = arr[0].length;

        int maxRow = -1;
        int j = m - 1;

        for(int i = 0; i < n; i++) {
            while(j >= 0 && arr[i][j] == 1) {
                j--;
                maxRow = i;
            }
        }

        return maxRow;
    }
}
```

Custom Input | Compile & Run | Submit